Software Engineering (Sessional) [CSE-355]



Proposal Report on:

BringIt – Peer to Peer Delivery

Name	Student ID
Mahmudul Hasan	2204040
Sakib Alif	2204051
Ayan Barua	2204053

Mir Md. Saki Kowsar
Professor
Department of CSE, CUET

Md. Refaj Hossan
Lecturer
Department of CSE, CUET

Department of Computer Science and Engineering (CSE)
Chittagong University of Engineering & Technology (CUET)
Chattogram-4349, Bangladesh

1 Introduction / Background

CUET's campus is located roughly 30 kilometers from Chittagong city, and a large share of the student body lives on-campus in residential halls. Everyday necessities that are not available near campus — stationery, medicine, specific food items, documents, or exam materials — require a trip into the city that many students cannot always make on short notice. At the same time, a significant number of students travel between the campus and the city daily for classes, errands, or personal reasons, and this travel capacity is almost entirely unused for anything beyond the traveler's own purpose. Commercial courier services such as Pathao or Sundarban exist, but their pricing and turnaround time are not designed for small, urgent, single-item deliveries between two known points inside a closed university community. BringIt addresses this gap by turning ordinary student travel into a trusted, low-cost peer delivery network restricted to verified CUET students.

2 Project Objectives

Main Objective

To design and develop an AI-assisted, web-based peer-to-peer delivery platform for the CUET campus community that connects students who need items from the city with students who are already traveling, making delivery faster, cheaper, and more trustworthy than existing alternatives.

Specific Objectives

- To build a request–traveler matching system that intelligently pairs delivery requests with travelers based on route, timing, and reliability, instead of a simple open feed.
- To introduce a fair, transparent reward suggestion so requesters do not need to guess a reasonable delivery fee.
- To establish a university-verified trust and reputation system that makes peer delivery safe for both requesters and travelers.
- To provide real-time status tracking of requests from posting to delivery, accessible on both desktop and mobile.
- To reduce dependency on commercial courier services for small, urgent, hyper-local deliveries within the CUET campus and Chittagong city.

3 Scope of the Project

In Scope

- Student authentication and verification using university ID/email.
- Posting, browsing, filtering, and accepting delivery requests.
- Rule-based matching of requests to suitable travelers.

- Rule-based reward/price suggestion at the time a request is posted.
- Delivery status tracking (Open → Accepted → In Progress → Delivered).
- Star-based rating and delivery history for trust scoring.
- Responsive web dashboard usable on mobile browsers.

Out of Scope

- Native mobile applications (iOS/Android) — the platform will be a responsive web app only in this phase.
- In-app payment processing — reward settlement will remain an off-platform arrangement (e.g., cash or mobile financial services) between requester and traveler.
- Delivery outside the CUET campus–Chittagong city corridor.

4 Proposed Features

- **Secure Authentication:** Registration and login restricted to verified CUET student IDs or university email addresses.
- **User Profiles:** Department- and hall-based profile information, delivery history, and credibility score.
- **Request Management:** Posting requests with pickup/drop-off locations, item description, urgency tagging (High/Medium/Low), and search/filtering by route.
- **Delivery Workflow:** Job acceptance by travelers and status tracking through Open, Accepted, In Progress, and Delivered states.
- **Trust & Reputation System:** Star-based ratings and a running credibility score built from delivery history.
- **Responsive Dashboard:** A mobile-friendly interface for real-time updates on active and past deliveries.

AI Integration :

- **Smart Traveler–Request Matching (Rule-Based Scoring):** Each open request is scored per traveler using a weighted formula:

  $$\text{Score} = w_1 \cdot \text{RouteOverlap} + w_2 \cdot \text{TimeWindowOverlap} + w_3 \cdot \text{UrgencyWeight} + w_4 \cdot \text{TravelerRating}$$

  where *RouteOverlap* measures how closely the pickup/drop-off points sit along the traveler's stated path, *TimeWindowOverlap* measures how well the request's timing fits the traveler's travel window, *UrgencyWeight* comes from the request's High/Medium/Low tag, and *TravelerRating* is the traveler's own credibility score. Requests are then ranked highest-score-first on that traveler's feed. This is a deterministic formula, not a trained model, so it works correctly from day one with no historical data.

- **Reward Price Suggestion (Rule-Based Formula):** When a requester posts a delivery, a formula suggests a starting reward:

  $$\text{Reward} = \text{BaseFee} + (\text{RatePerKm} \times \text{Distance}) + \text{UrgencyMultiplier} + \text{CategoryModifier}$$

  where *BaseFee* and *RatePerKm* are fixed constants, *UrgencyMultiplier* adds a premium for High-urgency requests, and *CategoryModifier* adds a small premium for fragile or bulky items. Like the matching score, this is a transparent formula rather than a trained model, so it needs no historical pricing data to work at launch. (Both formulas are intentionally left as a rule-based v1; if enough real usage data accumulates later, they could be upgraded to learned models, but that is future work, not part of this proposal.)

- **AI-Assisted Item Description Parsing (OpenAI API):** Both formulas above need structured inputs — item category, approximate size/fragility, and urgency — but requesters naturally type a free-text description like "need a scientific calculator for tomorrow's exam, handle carefully." Rather than forcing requesters to fill out multiple dropdowns for every post, the OpenAI API reads that one sentence and extracts the structured fields automatically (category: electronics, fragile: yes, urgency: high), which are then fed directly into the matching and pricing formulas. If a field cannot be confidently extracted, it falls back to a sensible default (e.g., urgency defaults to Medium) and is shown to the requester as an editable, pre-filled field rather than being silently submitted — so the system works correctly even when parsing gets nothing useful.

5 Methodology / Approach

Chosen SDLC Model

We will follow the Agile (Scrum) model, working in short sprints (approximately 1–2 weeks each) with sprint planning, daily stand-ups, sprint review, and retrospective. This suits the project because core delivery-workflow features can be delivered incrementally first, with the matching and pricing features layered on top once the request/traveler data model is stable, and feedback from each sprint can directly inform the next.

Technology Stack

- **Frontend:** React with TypeScript, mobile-responsive UI.
- **Backend:** FastAPI (Python) exposing REST endpoints for authentication, requests, matching, and pricing.
- **Database:** PostgreSQL via Supabase, used for structured data (users, requests, ratings) and auth.
- **Real-time updates:** WebSockets for live status changes on the activity dashboard.

AI Technology

- OpenAI API — the only AI service used; parses free-text delivery descriptions into structured item attributes (category, size/fragility, urgency).

- Matching and pricing themselves are rule-based formulas implemented directly in the FastAPI backend (no training data or ML library required for v1).
- Geopy / basic geospatial utilities for distance and route-overlap calculations feeding the matching and pricing formulas.

6 Expected Outcomes / Deliverables

- A final working web application implementing the full request-to-delivery workflow.
- **Key AI-powered features:** smart traveler–request matching and reward price suggestion, both demonstrably improving on a plain manual feed/guessed pricing.
- **Documentation:** a full SDLC report covering requirements, design (HLD/LLD/DB schema), sprint plan, testing, and deployment notes.

7 Tools & Resources Required

Hardware

- Development laptops/desktops (team-owned).
- Smartphones for testing the responsive/mobile experience.

Software / APIs / Cloud Services

- Supabase (managed PostgreSQL + auth) for the database layer.
- OpenAI API for structured parsing of item descriptions.
- GitHub for version control and Scrum-board-based task tracking.
- Free-tier cloud hosting (e.g., Vercel for frontend, Render/Railway for FastAPI backend) for deployment and demos.

8 Conclusion

BringIt reimagines the everyday movement of CUET students as a trusted, campus-restricted delivery network, closing a real gap left by expensive commercial couriers and the isolation of the campus from the city. What sets this iteration apart is its AI layer: rather than a passive request feed, the platform actively matches requests to the most suitable travelers and helps requesters set fair prices automatically, using transparent, explainable formulas that work correctly from day one. Combined with a verified-student trust model and a lightweight, mobile-first interface, BringIt has the potential to become a genuinely useful piece of everyday campus infrastructure while showcasing practical, well-scoped AI integration within a standard SDLC-driven build.